

Creating Arrays in NumPy

`np.array()` | `np.arange()`

NumPy gives us more than one way to create an ndarray. Two of the most common are converting an existing Python list with **`np.array()`**, and generating a sequence of numbers directly with **`np.arange()`**.

1. Creating a NumPy Array from a List — `np.array()`

1.1 Syntax

```
script.py  
  
np.array([ele1, ele2, ele3, ...])
```

1.2 Example

```
script.py  
  
import numpy as np  
  
num_arr = np.array([1, 2, 4, 6, 7, 8])  
  
print(num_arr) # it will print a numpy array
```

1.3 Use case

Use `np.array()` when you already have a Python list holding a large amount of data, and you need to perform mathematical operations on it. Converting the list to a NumPy array lets those operations run fast and directly, instead of writing manual loops.

Note: `np.array()` takes an existing Python list (or list of lists, for 2-D data) and copies its values into a new, single-typed ndarray. The original list is left unchanged.

2. Creating a Sequence — np.arange()

The **arange()** function is used to create a 1-D NumPy array containing a sequence of numbers.

2.1 Syntax

```
script.py

np.arange(start, stop, step)
```

Note: start is included, but stop is excluded — just like Python's range(). step controls the gap between consecutive numbers (default is 1).

2.2 Examples

```
script.py

import numpy as np

# sequence 1: step = 1
num_py1 = np.arange(1, 10, 1)
print(num_py1)
# Output: [1 2 3 4 5 6 7 8 9]

# sequence 2: step = 2
num_py2 = np.arange(1, 10, 2)
print(num_py2)
# Output: [1 3 5 7 9]

# sequence 3: step = 3
num_py3 = np.arange(1, 10, 3)
print(num_py3)
# Output: [1 4 7]
```

2.3 Use case

Use np.arange() when you want to store a large amount of numeric data following a regular sequence, without typing out every value manually.

Quick Summary

Function	Purpose	Input
np.array()	Convert an existing list into a NumPy array	A Python list
np.arange()	Generate a sequence of numbers directly	start, stop, step